I2C Master/Slave IP Core

with APB Register Interface, 10-bit Addressing,
Clock Stretching and Multi-Master Arbitration

Design and Verification Report

B. Srishanth Reddy

Electronics and Communication Engineering

November 2026

Tools: Verilog-2005 RTL · Icarus Verilog 12 · GTKWave · Yosys 0.52

Abstract

This report describes an I2C (Inter-Integrated Circuit) master and slave IP core that I designed in synthesizable Verilog and verified with self-checking testbenches. The master handles START, repeated START and STOP, 7- and 10-bit addressing, reads and writes with ACK/NACK, a programmable SCL prescaler for 100 kHz and 400 kHz, and detects lost arbitration when two masters talk at once. The slave has a programmable address, a 16 x 8 register file and clock stretching. An AMBA APB wrapper lets a CPU drive the master through six registers. Three testbenches with a real open-drain bus model run 15 tests and 91 checks, and all of them pass. A deliberately injected bug is caught by the testbench. Waveforms were checked in GTKWave, and the screenshots in this report are real captures of the laptop screen.

1. Introduction

I2C is a two-wire serial bus (SCL clock, SDA data) used to connect sensors, EEPROMs, RTCs and PMICs to a processor. Both lines are open-drain: a device can only pull a line low, and a pull-up resistor makes it high. This wired-AND behaviour is what allows several devices, and even several masters, to share the bus. After my Configurable UART IP Core, I wanted to build a protocol that has addressing, acknowledgements and multi-master behaviour, so I chose I2C.

Objectives:

- Synthesizable master and slave in Verilog, connected through a real open-drain bus model
- Master: START / repeated START / STOP, 7- and 10-bit addressing, read/write, ACK/NACK, 100k/400k prescaler, arbitration-loss detection
- Slave: programmable address, register file, clock stretching
- APB register interface so the core can be used in an SoC
- Self-checking testbenches for every transfer type, NACK, stretching and arbitration, with waveforms in GTKWave

2. I2C protocol basics

Condition	Definition
START (S)	SDA goes high to low while SCL is high
STOP (P)	SDA goes low to high while SCL is high
Repeated START (Sr)	A START without a STOP before it; used to change direction (write pointer, then read)
Data bit	SDA changes only while SCL is low and is sampled while SCL is high, MSB first
ACK / NACK	9th bit: the receiver pulls SDA low (ACK) or leaves it high (NACK)
7-bit address	First byte = A6..A0 + R/W
10-bit address	First byte = 11110 A9 A8 R/W, second byte = A7..A0
Clock stretching	A slave holds SCL low to make the master wait
Arbitration	A master that sends 1 but reads 0 on SDA has lost and must release the bus

3. Architecture

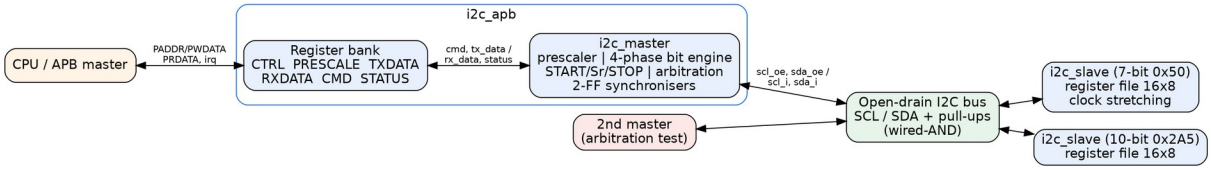


Figure 1: Block diagram of the IP core and the verification setup

The design has three modules. i2c_master is the bus engine, i2c_slave is a register-file slave, and i2c_apb wraps the master with an APB register bank. Every SCL/SDA driver is open-drain: the output enable pulls the line low and otherwise leaves it at 'z'. In the testbenches, the bus wires are declared tri1, which models the pull-up resistors.

3.1 Bit timing

Each bit, START and STOP is split into four phases. Each phase lasts PRESCALE+1 clock cycles, so f_SCL = f_clk / (4 x (PRESCALE + 1)). At 50 MHz, PRESCALE = 124 gives 100 kHz and PRESCALE = 30 gives 403 kHz.

Phase	SCL	SDA / action
ph0	driven low	master puts the next bit on SDA (or releases it to read)
ph1	released	the counter is held at 0 until SCL is really high; this supports clock stretching and clock synchronisation between masters. SDA is sampled and arbitration checked at the end of ph1
ph2	high	START: SDA pulled low here; STOP: SDA released here
ph3	driven low	end of bit

3.2 Master FSM

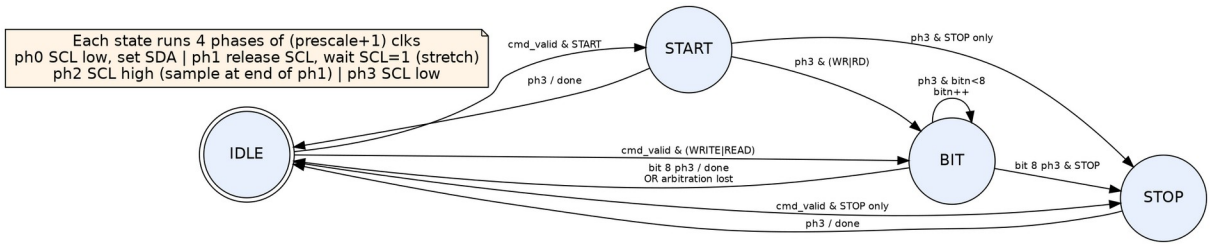


Figure 2: Master state machine

A command can contain START, one WRITE or READ byte and STOP, so a whole byte transfer needs one command. During a WRITE, if the master releases SDA (sends 1) but the synchronised SDA is 0 at the sampling point, another master is driving the bus. The master then sets arb_lost, releases both lines in the same cycle and goes back to IDLE. A START/STOP monitor drives bus_busy, so software can wait for a free bus before retrying.

3.3 Slave FSM

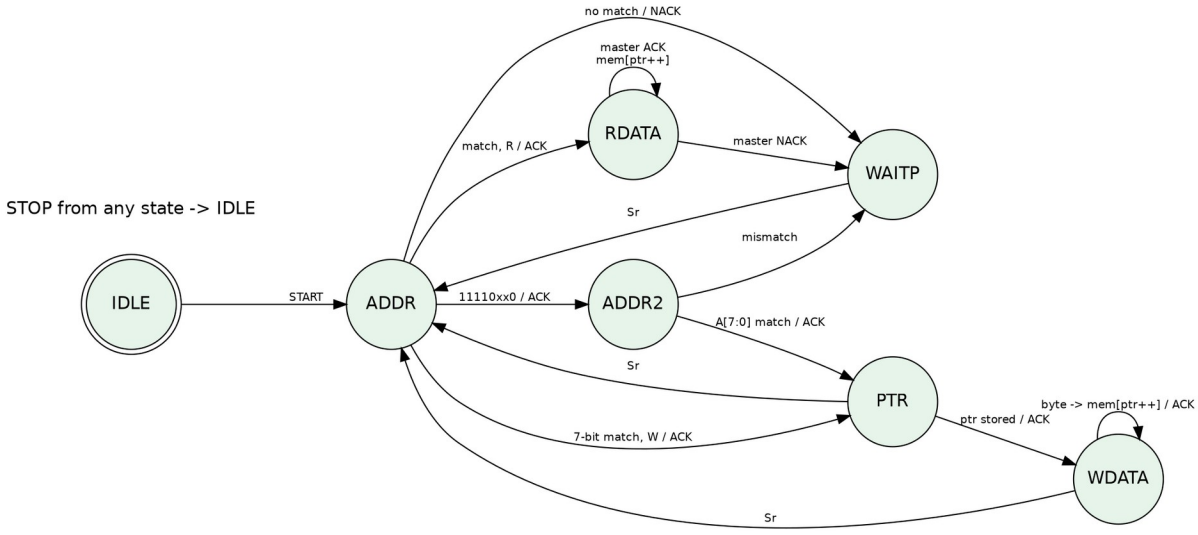


Figure 3: Slave state machine

The slave detects START and STOP from synchronised SCL/SDA. It shifts data in on SCL rising edges and changes SDA only after SCL falling edges. The first byte after the address is the register pointer, and it increments after every data byte. For 10-bit mode, the slave answers the header 11110 A9 A8 0, checks the low address byte, and remembers the match, so a later Sr with 11110 A9 A8 1 starts a read. When clock stretching is enabled, the slave pulls SCL low for stretch_cycles clocks after each ACK bit.

3.4 APB register map

Offset	Name	Access	Description
0x00	CTRL	RW	[0] irq_en
0x04	PRESCALE	RW	[15:0] SCL prescaler, reset 124
0x08	TXDATA	RW	[7:0] byte to send
0x0C	RXDATA	RO	[7:0] last byte read
0x10	CMD	WO	[0] START [1] STOP [2] READ [3] WRITE [4] NACK; a write starts the command
0x14	STATUS	RO/W1C	[0] busy [1] rx_nack [2] arb_lost [3] done (W1C) [4] bus_busy

4. Verification

All three testbenches are self-checking. Each test compares register contents, received bytes, ACK/NACK flags and measured SCL timing against expected values, and prints PASS or FAIL. scripts/run_all.ps1 compiles and runs everything with Icarus Verilog on Windows and writes the logs. To make sure the testbench can really detect errors, a negative test builds the slave with +define+INJECT_BUG, which flips bit 0 of every byte written to the register file. The core testbench must then fail, and it does.

Testbench	Test	Result
tb_i2c_core	1. 7-bit write, 4 bytes, 100 kHz	PASS

tb_i2c_core	2. 7-bit read, repeated START, ACK/NACK	PASS
tb_i2c_core	3. NACK: no slave at address 0x58	PASS
tb_i2c_core	4. 10-bit write to 0x2A5	PASS
tb_i2c_core	5. 10-bit read with repeated START	PASS
tb_i2c_core	6. 10-bit NACK: wrong low address byte	PASS
tb_i2c_core	7. clock stretching by slave (600 clk = 12 us)	PASS
tb_i2c_core	8. 400 kHz fast mode (prescale=30)	PASS
tb_i2c_arb	1. arbitration lost detected, winner completes	PASS
tb_i2c_arb	2. retry after arbitration loss	PASS
tb_i2c_arb	3. arbitration in data phase	PASS
tb_i2c_apb	1. APB register access	PASS
tb_i2c_apb	2. APB write transfer	PASS
tb_i2c_apb	3. APB read transfer	PASS
tb_i2c_apb	4. NACK status	PASS
negative	INJECT_BUG: corrupted register write	Bug caught

Summary lines from logs/summary.log:

```
SUMMARY tb_i2c_core: 8/8 tests PASS, 55 checks, 0 errors
RESULT: ALL TESTS PASSED
SUMMARY tb_i2c_arb: 3/3 tests PASS, 17 checks, 0 errors
RESULT: ALL TESTS PASSED
SUMMARY tb_i2c_apb: 4/4 tests PASS, 19 checks, 0 errors
RESULT: ALL TESTS PASSED
NEGATIVE TEST: injected bug detected by testbench ... PASS
```

Measured values from the logs:

- SCL period min 10060.0 ns max 10120.0 ns -> 99.40 kHz
- longest SCL low time 12060.0 ns (normal low = 5000 ns)
- SCL period min 2540.0 ns max 2600.0 ns -> 393.70 kHz
- master A done at 9090 ns, arb_lost=1
- master B done at 25570 ns, arb_lost=0, rx_nack=0
- data-phase: A arb_lost=1, B arb_lost=0

5. Waveforms (GTKWave screenshots)

The figures below are real screenshots of the laptop screen. GTKWave opened the VCD files from the simulation runs, and scripts/gtkwave_shots.ps1 captured each view. The view scripts in scripts/gtkw/ set the signals, zoom and marker.

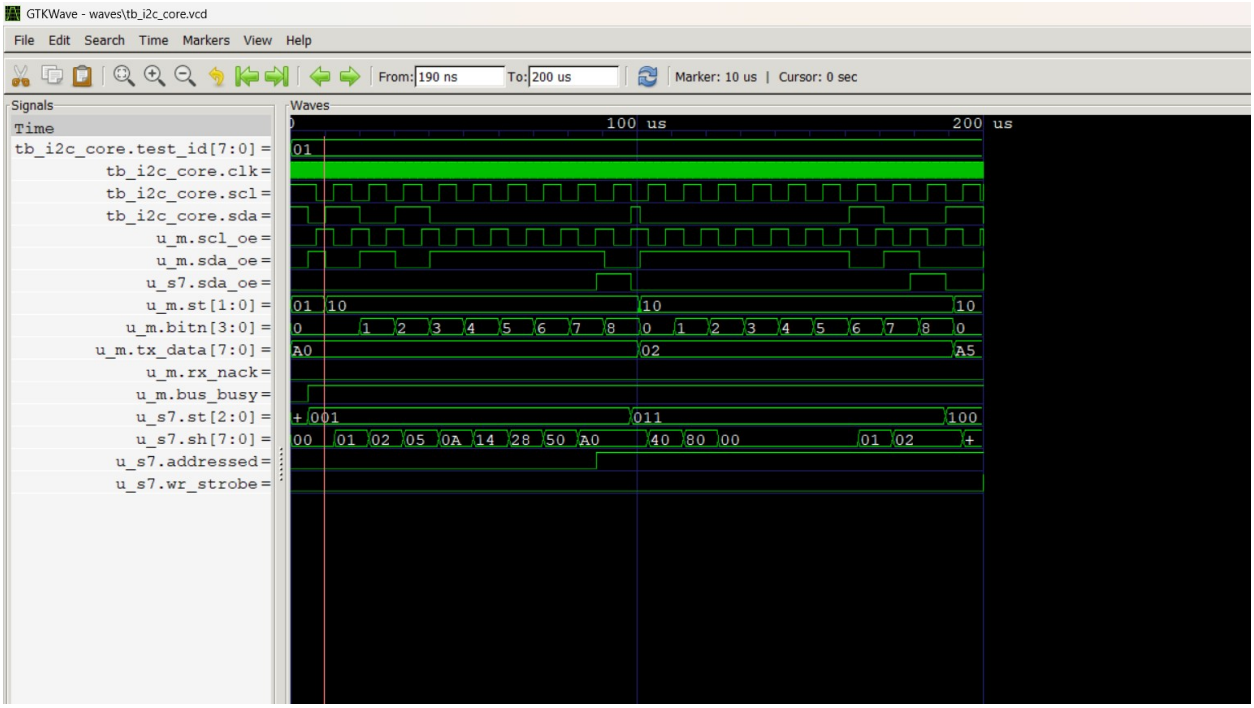


Figure 4: 7-bit write at 100 kHz: START, address 0xA0 + ACK, pointer, data bytes (tb_i2c_core test 1)

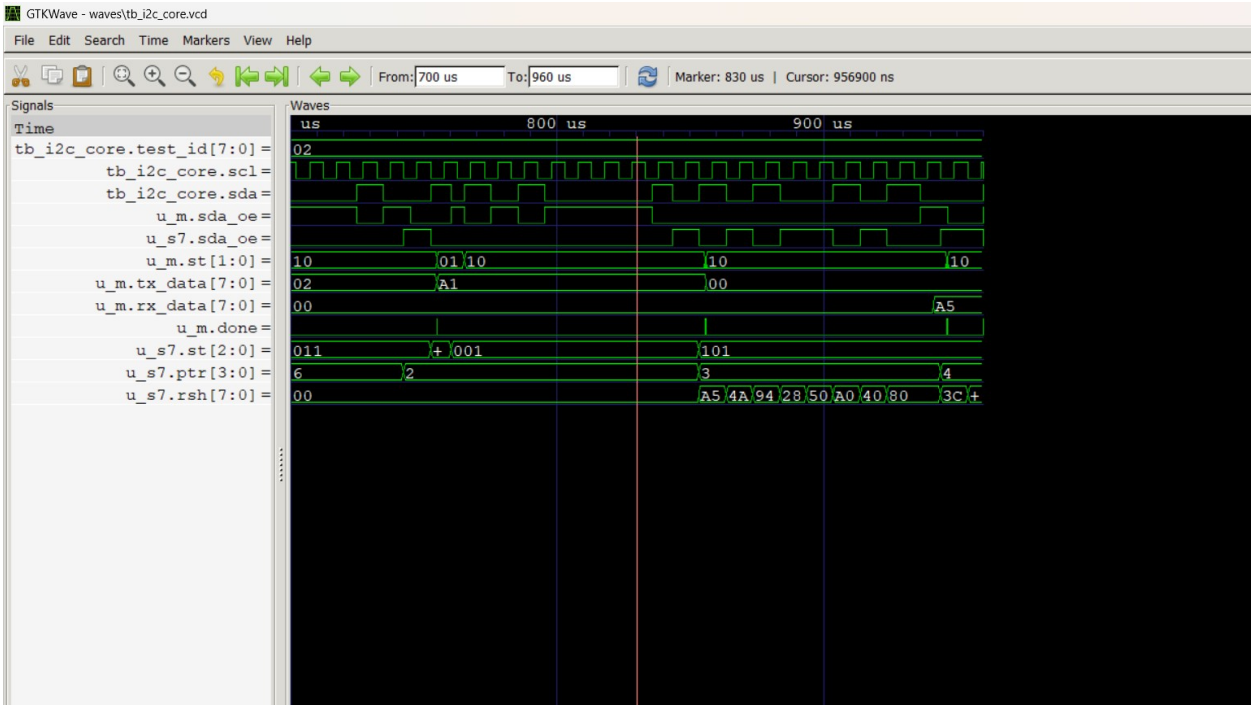


Figure 5: Read with repeated START: the slave drives data, the master ACKs and NACKs the last byte (test 2)

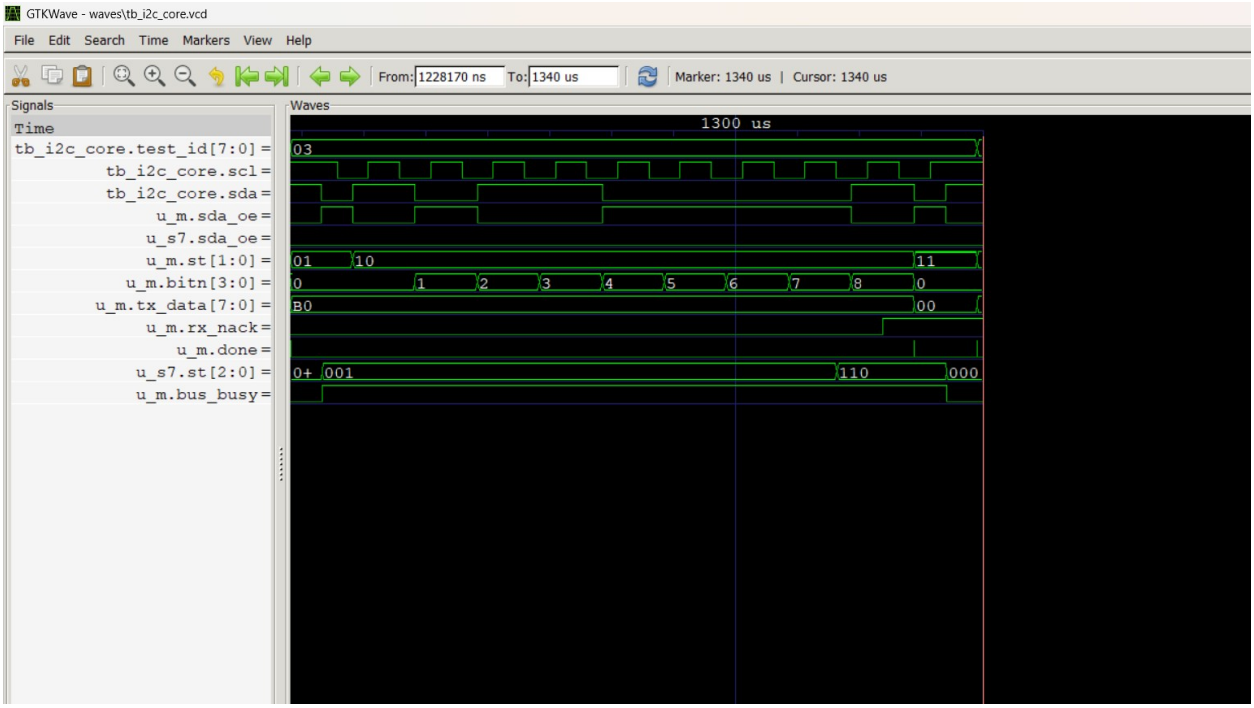


Figure 6: Address NACK: nobody answers 0x58, so SDA stays high on the 9th bit and rx_nack = 1 (test 3)

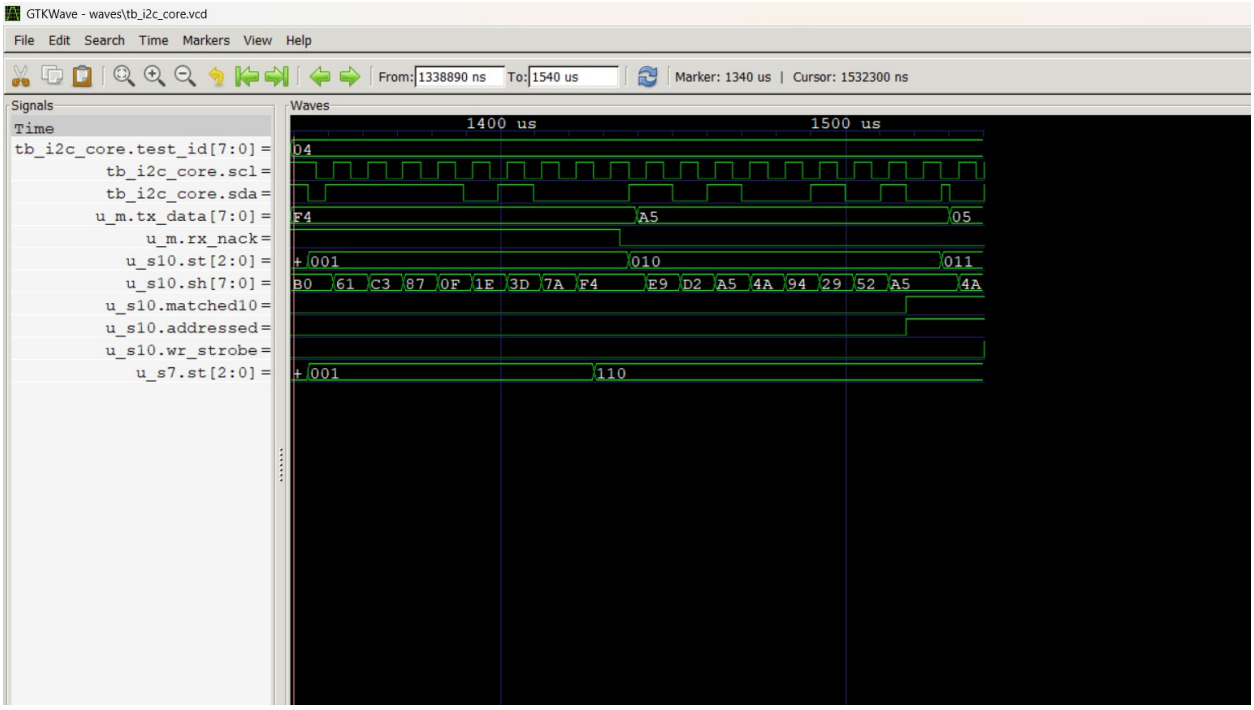


Figure 7: 10-bit addressing: header 0xF4 then low byte 0xA5; the 10-bit slave sets matched10 (test 4)

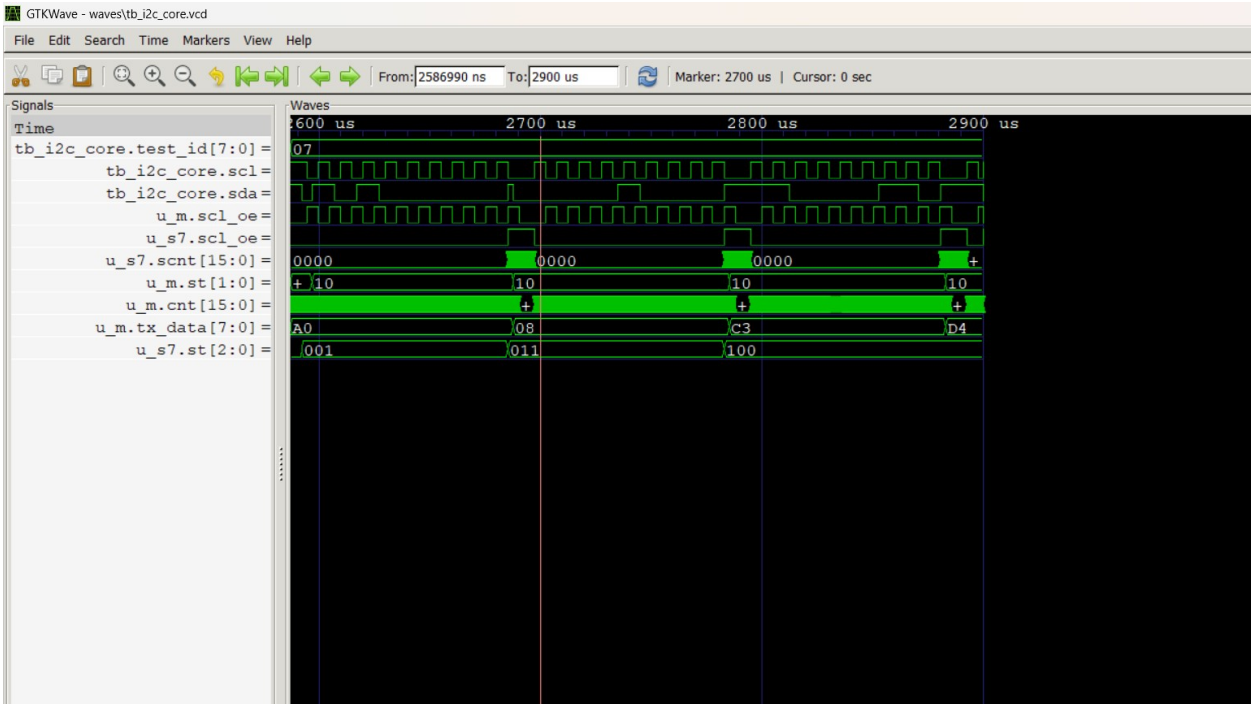


Figure 8: Clock stretching: the slave holds SCL low (scl_oe, scnt) and the master waits in phase 1 (test 7)

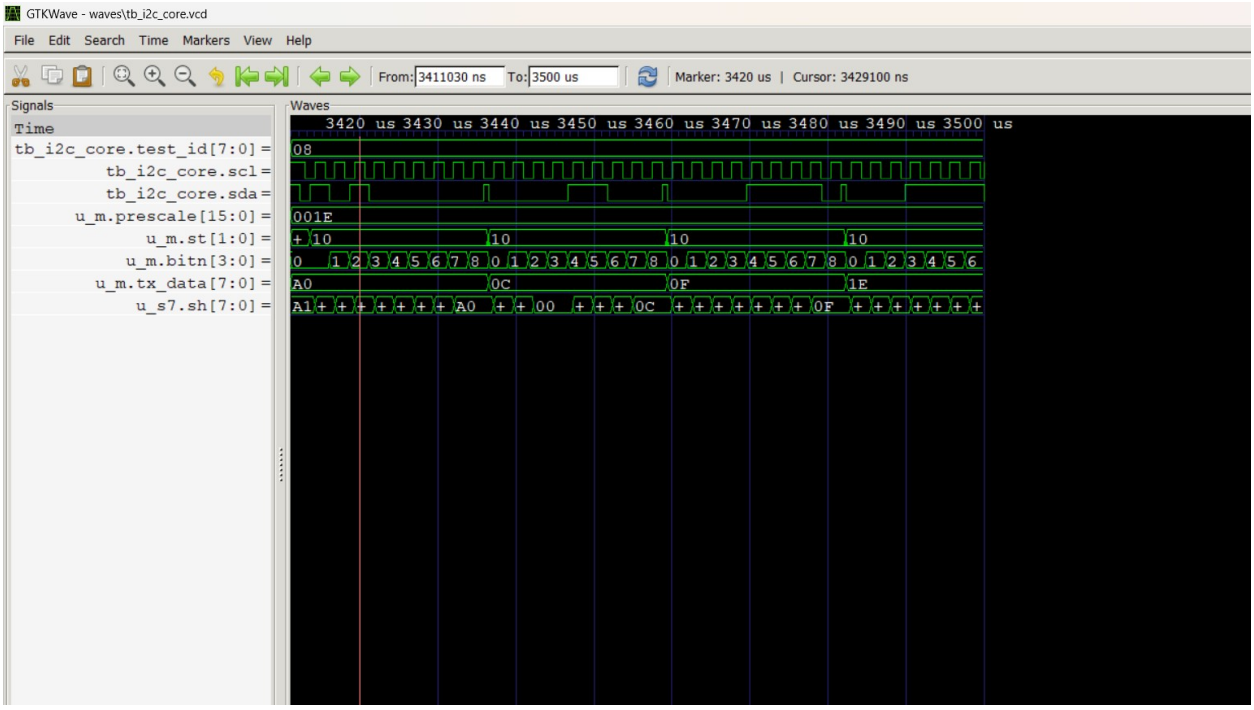


Figure 9: Fast mode, 400 kHz with PRESCALE = 30 (test 8)

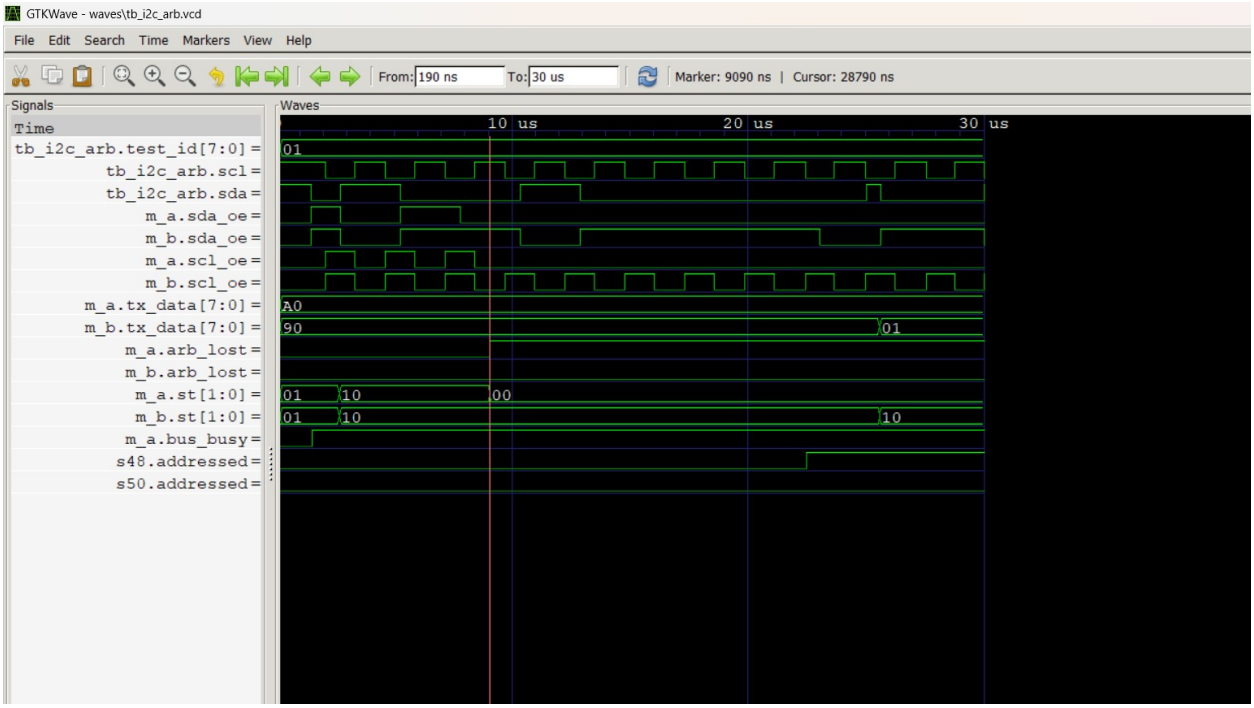


Figure 10: Multi-master arbitration: A sends 0xA0, B sends 0x90; A reads 0 while sending 1, sets arb_lost and releases the bus

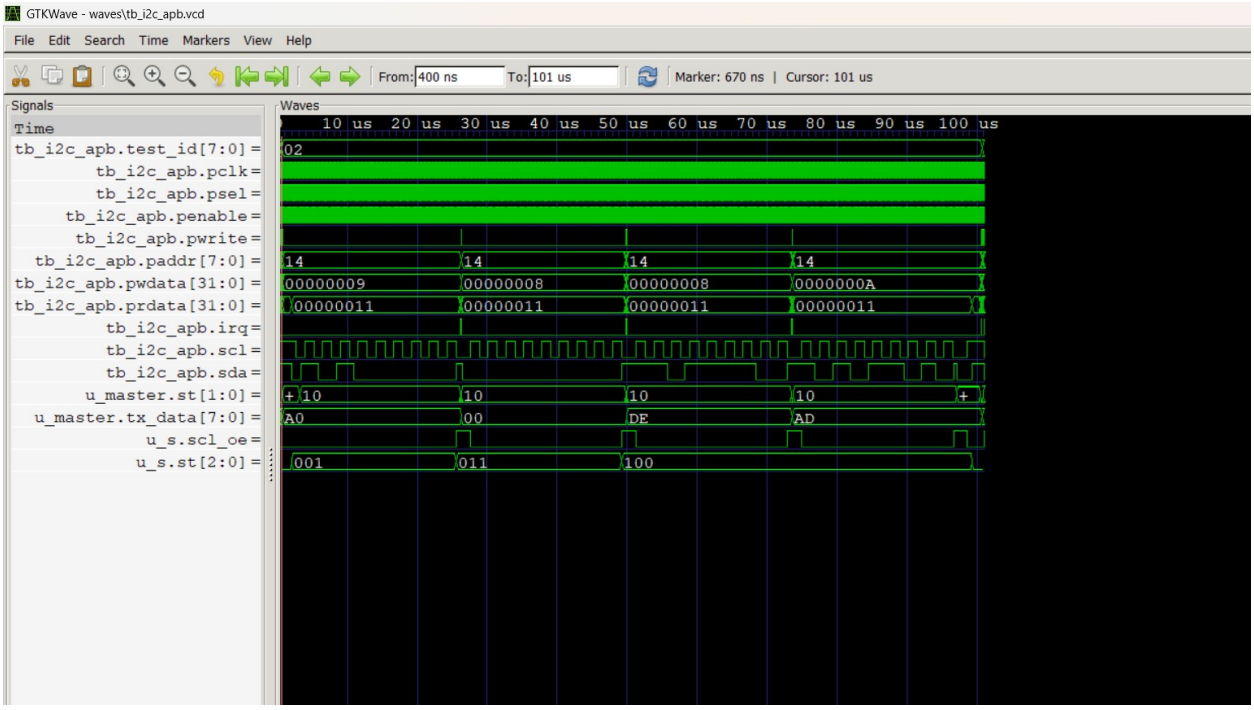


Figure 11: APB-driven transfer: APB writes to TXDATA/CMD, STATUS polling, irq, and the I2C bus with slave stretching

6. Synthesis

Top	Generic cells	Flip-flops
i2c_master	399	57
i2c_slave (16 x 8 regs)	1293	186

i2c_apb (wrapper + master)	544	89

Yosys 0.52 'synth -flatten' with generic gates (syn/synth.ys). This shows the RTL is synthesizable; it is not a real cell library, so there are no area or timing figures. Yosys is not installed on the laptop, so this step ran separately on Linux.

7. Results and discussion

- All 15 tests in 3 testbenches pass with 0 errors over 91 checks, and the injected bug is detected.
- Measured SCL is 99.40 kHz at PRESCALE = 124 and 393.70 kHz at PRESCALE = 30. It is slightly below the ideal 100/403 kHz because of the 2-flop synchroniser delay in phase 1.
- With stretching on, the longest SCL low time was 12.06 µs against 5 µs normally, and every byte still transferred correctly. This shows the master waits for SCL.
- In arbitration, the losing master releases the bus in the same bit, and the winner finishes without any corruption.

8. Limitations and future work

- Byte-level master: software sends the address bytes; a transaction-level sequencer, FIFOs and DMA would reduce CPU load.
- No spike filter on SCL/SDA, and only one prescaler (no separate tLOW/tHIGH/setup/hold registers).
- Arbitration is not checked during the START condition itself.
- Simulation-only verification; next steps are UVM-style random tests, SVA assertions, formal checks and an FPGA test with a real I2C sensor.

9. Conclusion

I designed and verified a complete I2C master/slave IP core with an APB interface. It covers the key parts of the I2C specification: START/Sr/STOP, 7- and 10-bit addressing, ACK/NACK, standard and fast mode, clock stretching and multi-master arbitration. Every result in this report comes from the simulation logs and real GTKWave screenshots.

References

- NXP Semiconductors, UM10204 I2C-bus specification and user manual, Rev. 7.0, 2021.
- ARM, AMBA 3 APB Protocol Specification v1.0, 2004.
- S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd ed., Pearson, 2003.